GPS Enabled Food Delivery Bot

ABSTRACT

With the rapid growth of automation and smart logistics, autonomous delivery systems are gaining significant importance. This project presents the design and implementation of a **GPS-enabled autonomous food delivery robot** capable of navigating to a predefined outdoor destination while avoiding obstacles in real time.

The robot uses a **NEO-8M GPS module** for position tracking, a **BMM150 magnetometer** for orientation and directional correction, and **ultrasonic sensors** for obstacle detection. An **ESP32 microcontroller** acts as the central control unit, processing sensor data and controlling the motors through an **L298N motor driver**. The destination coordinates are transmitted via a mobile application.

Unlike commercial systems, this prototype uses a **passive food container**—a packed plastic box mounted on the chassis—making the design simple, lightweight, and cost-effective. The robot autonomously travels to the destination point and stops once it reaches the target coordinates.

The proposed system reduces human involvement, improves delivery efficiency, and demonstrates a practical low-cost solution for short-distance food delivery in environments such as campuses, hospitals, and industrial areas.

CHAPTER 1 – INTRODUCTION

1.1 Background

Food delivery services have become an essential part of modern society. Traditional delivery methods rely heavily on human labor, which can be inefficient, costly, and unsafe in certain environments such as hospitals, industrial campuses, and restricted zones. The growing demand for contactless and automated services has created the need for autonomous delivery systems.

Autonomous robots equipped with navigation and sensing capabilities provide a promising alternative. With the integration of GPS, wireless communication, and real-time obstacle detection, robots can independently navigate outdoor environments and deliver items with minimal human intervention.

1.2 Problem Statement

Conventional food delivery methods face several challenges:

- Dependence on human workforce
- High operational cost
- Safety risks in restricted or hazardous areas
- Inefficiency for short-distance delivery

There is a need for a low-cost autonomous system that can deliver food over short distances without human involvement.

1.3 Motivation

The motivation behind this project is to develop a simple, affordable, and reliable autonomous delivery robot that can:

- Navigate using GPS
- Avoid obstacles dynamically
- Operate with minimal human control
- Demonstrate real-world automation concepts

This project also serves as a learning platform to integrate embedded systems, sensors, motor control, and mobile applications.

1.4 Objectives

The objectives of the project are:

- To design a GPS-enabled autonomous robot
- To implement real-time obstacle avoidance
- To control robot movement using a microcontroller
- To enable destination input via a mobile application
- To ensure efficient power management
- To demonstrate autonomous delivery using a passive food container

CHAPTER 2 – SYSTEM ARCHITECTURE

2.1 Overall System Architecture

The GPS Enabled Autonomous Food Delivery Bot is designed as an embedded robotic system that integrates navigation, sensing, control, and actuation modules into a single autonomous platform. The system follows a centralized control architecture in which the **ESP32 microcontroller** acts as the core processing unit.

All peripheral modules—including GPS, magnetometer, ultrasonic sensor, and motor driver—interface with the ESP32. The microcontroller continuously gathers data from the sensors, processes it, and generates control signals for the motors. Destination coordinates are provided through a mobile application, enabling wireless communication between the user and the robot.

The overall architecture can be divided into five major functional units:

1. **User Interface Module** – Android Mobile App
2. **Navigation Module** – GPS + magnetometer
3. **Obstacle Detection Module** – Ultrasonic Sensor
4. **Control Unit** – ESP32 Microcontroller
5. **Actuation Module** – Motor Driver and DC Motors
6. **Power Supply Unit** – Battery Pack and Buck Converter

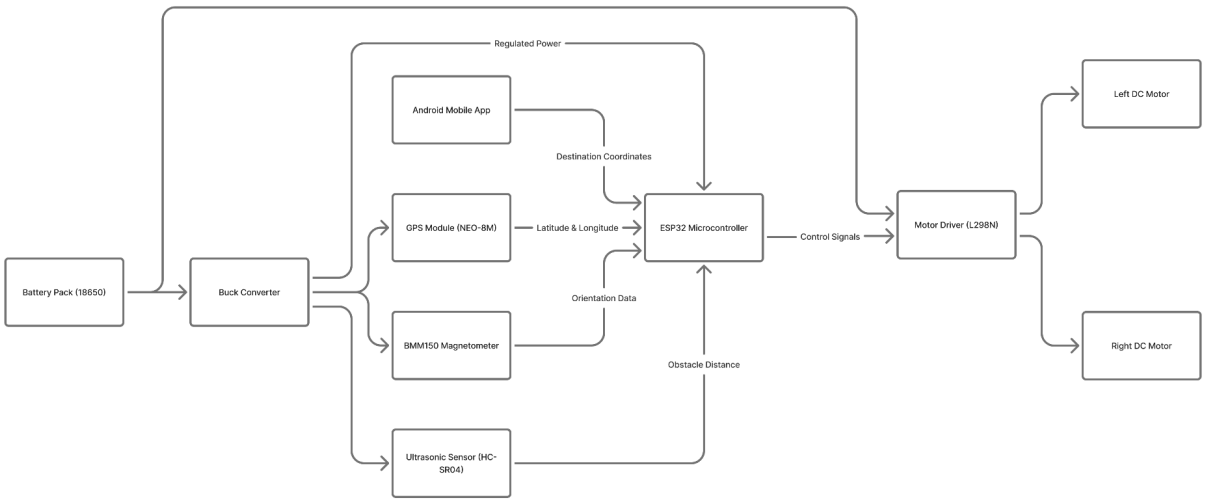
Each unit performs a specific function while working in coordination with the others to achieve autonomous navigation and delivery.

2.2 Block Diagram Description

The block diagram represents the functional flow of the entire system.

- The **Android Mobile Application** sends the destination latitude and longitude to the ESP32 over Wi-Fi.
- The **NEO-8M GPS module** continuously provides real-time geographical coordinates of the robot.
- The **magnetometer(BMM150)** supplies orientation and directional data, enabling heading correction.
- The **Ultrasonic Sensor (HC-SR04)** detects obstacles in the robot's path and sends distance data to the controller.
- The **ESP32 Microcontroller** compares the current GPS position with the target coordinates and determines the direction and movement strategy.

- Based on this computation, control signals are sent to the **L298N Motor Driver**, which drives the left and right DC motors.
- The robot moves toward the destination while dynamically avoiding obstacles.
- Power is supplied by **18650 Li-ion batteries**, and a **buck converter** regulates voltage for stable operation of all modules.



This architecture ensures modularity, allowing each subsystem to be upgraded independently.

2.3 Hardware Architecture

The hardware architecture defines how physical components are interconnected and how data flows between them.

2.3.1 Control Unit – ESP32

The ESP32 is the heart of the system. It performs the following tasks:

- Receives destination coordinates from the mobile app
- Reads GPS data and current location
- Processes magnetometer orientation values
- Monitors obstacle distance
- Executes navigation and avoidance algorithms
- Generates motor control signals

Its built-in Wi-Fi capability eliminates the need for external communication modules.

2.3.2 Navigation Module

- **NEO-8M GPS**: Provides real-time latitude and longitude. Used to determine the robot's current position.
- **Magnetometer(BMM150)**: Measures heading direction of the bot. Helps maintain correct heading toward the destination.

Together, these sensors enable outdoor navigation and directional correction.

2.3.3 Obstacle Detection Module

- **HC-SR04 Ultrasonic Sensor**: Emits ultrasonic waves and measures echo time to calculate distance to obstacles.
  If an object is detected within a predefined threshold, the robot alters its path.

2.3.4 Motor Drive System

- **L298N Motor Driver**: Acts as an interface between the low-power microcontroller and high-current DC motors. Allows forward, reverse, and differential steering.
- **DC Motors and Wheels**: Provide locomotion. By varying the speed and direction of each motor, the robot can turn and maneuver.

2.3.5 Food Container Module

The robot carries a **passive food container**—a packed plastic box mounted on the chassis. There is no mechanical opening or closing mechanism. The robot's role is limited to transporting the container and stopping at the destination point.

2.3.6 Power Supply Unit

- **18650 Li-ion Batteries (3.7V each)** provide portable energy.
- **Buck Converter** steps down and regulates voltage to safe levels for ESP32, sensors, and motor driver.

This ensures:

- Stable operation
- Protection against voltage fluctuations
- Extended battery life

2.4 Software Architecture

The software architecture is divided into three logical layers:

1. **Application Layer**
    - Android mobile app developed using MIT App Inventor
    - Allows the user to input destination coordinates
    - Sends data wirelessly to the robot
2. **Control Layer (ESP32 Firmware)**
    - Receives coordinates
    - Reads GPS, magnetometer, and ultrasonic data
    - Executes navigation and obstacle avoidance algorithms
    - Generates motor control commands
3. **Hardware Abstraction Layer**
    - Sensor interfacing routines
    - Motor driver control functions
    - Serial communication drivers

This layered approach improves clarity, debugging, and future scalability.

CHAPTER 3 – HARDWARE DESIGN

This chapter describes all the hardware components used in the GPS Enabled Autonomous Food Delivery Bot. Each component is selected based on cost, availability, ease of integration, and suitability for autonomous robotic applications.

3.1 ESP32 Microcontroller

The ESP32 serves as the central processing unit of the system.

Key Features

- 32-bit microcontroller
- Built-in Wi-Fi module
- Operates at 5V
- Multiple GPIO pins
- UART, I2C, and SPI support

Role in the System

- Receives destination coordinates from the mobile app
- Reads data from GPS, magnetometer, and ultrasonic sensor
- Processes navigation logic
- Controls the motor driver
- Manages obstacle avoidance

The ESP32 eliminates the need for an external communication module, reducing system complexity and cost.

3.2 NEO-8M GPS Module

The NEO-8M is a widely used GPS receiver module for embedded systems.

Specifications

- Operating Voltage: 3.3V – 5V
- Communication: UART
- Position Accuracy: ~2.5 meters
- Update Rate: 1–10 Hz

Role in the System

- Provides real-time latitude and longitude
- Tracks the robot's current position
- Enables outdoor navigation

The ESP32 continuously compares GPS data with the target coordinates to determine movement direction.

3.3 MAGNETOMETER(BMM150)

The BMM150 serves as a 3-axis magnetometer sensor.

Specifications

- Operating Voltage: 3.3V
- Interface: I2C
- Measures acceleration and angular velocity

Role in the System

- Acts as a digital compass, measuring the Earth's magnetic field to determine the robot's absolute heading (yaw).
- The BMM150 provides reliable orientation data used to decide left/right turns and alignment.
- It enhances navigation accuracy, especially during turns and obstacle avoidance maneuvers.

3.4 Ultrasonic Sensor (HC-SR04)

The HC-SR04 is used for obstacle detection.

Specifications

- Operating Voltage: 5V
- Measuring Range: 2 cm – 400 cm
- Accuracy: ±3 mm

Role in the System

- Continuously scans the path ahead
- Detects obstacles
- Sends distance data to ESP8266
- Triggers avoidance behavior when an object is within threshold distance

This ensures safe movement in dynamic environments.

3.5 Motor Driver Module – L298N

The L298N is a dual H-bridge motor driver IC.

Specifications

- Motor Voltage: Up to 40V
- Logic Voltage: 12V
- Can drive four DC motors

Role in the System

- Amplifies low-power signals from ESP32
- Controls direction and speed of motors

- Enables forward, reverse, left, and right movement

It protects the microcontroller from high current drawn by motors.

3.6 DC Motors, Wheels, and Chassis

- **DC Motors** provide mechanical movement
- **Wheels** enable smooth locomotion
- **Chassis** forms the physical structure of the robot

The motors are connected to the L298N motor driver. Differential speed control enables turning and navigation.

3.7 Power Supply System

Components

- 18650 Li-ion Batteries x 2 (3.7V each)
- Buck Converter(AI2148)

Role

- Batteries provide portable power
- Buck converter regulates voltage for:
    - ESP32
    - Ultrasonic sensor
    - Motor driver logic

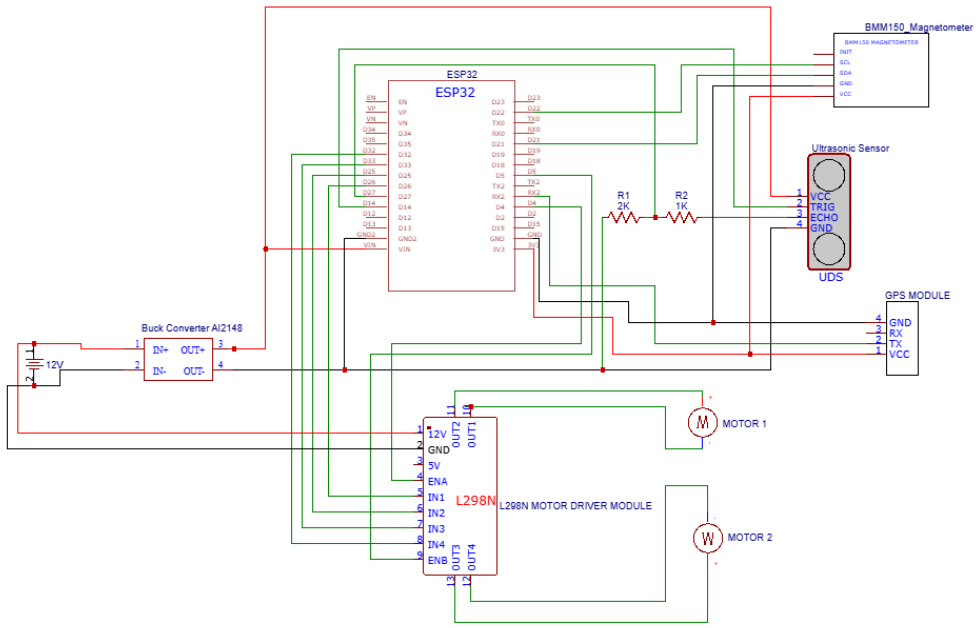
This ensures stable and efficient operation of all modules.

3.8 Passive Food Container

A packed plastic box is mounted on the chassis to carry food items.

- No servo or mechanical actuator is used
- Lightweight and simple design
- Suitable for prototype demonstration
- Reduces mechanical complexity

The robot's function is to transport and stop at the destination, where the food can be collected manually.



CHAPTER 4 – SOFTWARE DESIGN

The software design of the GPS Enabled Autonomous Food Delivery Bot is divided into two major parts:

1. **Mobile Application Software**
2. **Embedded Firmware for ESP32**

The software is responsible for user interaction, data communication, sensor processing, navigation logic, and motor control.

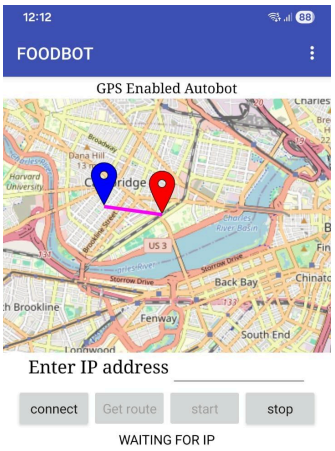
4.1 Mobile Application Design

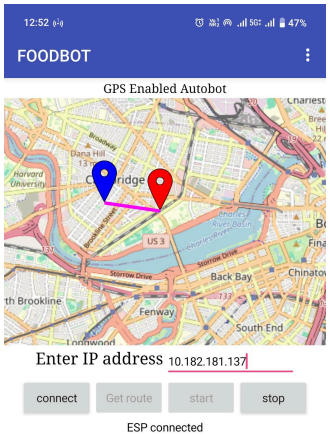
The mobile application is developed using **MIT App Inventor**. It serves as the user interface for the system.

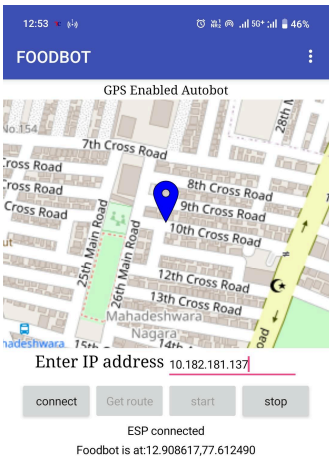
Features of the Mobile App

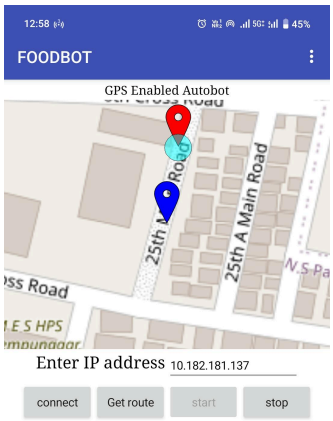
- Input fields for ESP32 IP address.
- Button to connect to the microcontroller.

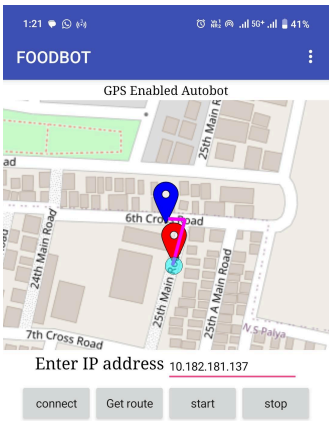
- Button ‘get route’ to fetch the route from source to destination from the OpenRouteService platform.
- Wi-Fi-based communication
- Simple and user-friendly interface

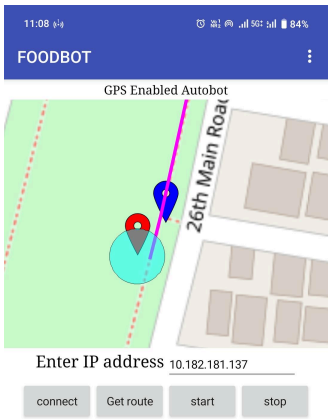












Functionality

1. The user enters the IP address and selects the destination coordinates by tapping on the map.
2. On pressing the 'get route' button, the app gets the waypoints from the OpenRouteService platform and transmits the data to the ESP32.
3. The robot acknowledges receipt and begins navigation once the 'start' button is pressed.

This eliminates the need for manual programming for each mission and allows flexible destination selection.

4.2 ESP32 Firmware Architecture

The firmware running on the ESP32 is written using the **Arduino IDE**.

It is structured into the following functional modules:

- Wi-Fi Communication Handler
- GPS Data Parser
- MagnetometerData Reader
- Obstacle Detection Routine
- Navigation Algorithm
- Motor Control Driver

Each module operates in coordination within the main program loop.

4.3 Communication Protocol

- Communication between mobile app and ESP32 occurs over Wi-Fi.
- Destination coordinates are transmitted as text strings:
  LAT:xx.xxxxxx, LON:yy.yyyyyy
- The ESP32 parses this data and stores it as the target location.

This lightweight protocol is efficient and easy to debug.

4.4 Sensor Handling Logic

GPS Handling

- Data is received over UART
- NMEA sentences are parsed
- Latitude and longitude are extracted

- Updated at regular intervals

Magnetometer Handling

- Orientation data is read using I2C
- Used to determine heading
- Helps in direction correction

Ultrasonic Handling

- Trigger pulse is sent
- Echo time is measured
- Distance is calculated using: Distance = (Time × Speed of Sound) / 2
- If distance < threshold, obstacle is detected

4.5 Motor Control Logic

Motor movement is controlled using digital outputs from ESP32:

Condition	Left Motor	Right Motor
Move Forward	Forward	Forward
Turn Left	Stop/Slow	Forward
Turn Right	Forward	Stop/Slow
Stop	Stop	Stop
Obstacle Detected	Right	Right

PWM signals may be used for speed control.

4.6 Error Handling and Safety

- If GPS signal is lost, robot stops
- If obstacle is continuously detected, robot halts
- If Wi-Fi connection drops, last known target is retained
- Battery voltage can be monitored to prevent deep discharge

This ensures safe and predictable behavior.

CHAPTER 5 – WORKING PRINCIPLE AND ALGORITHM

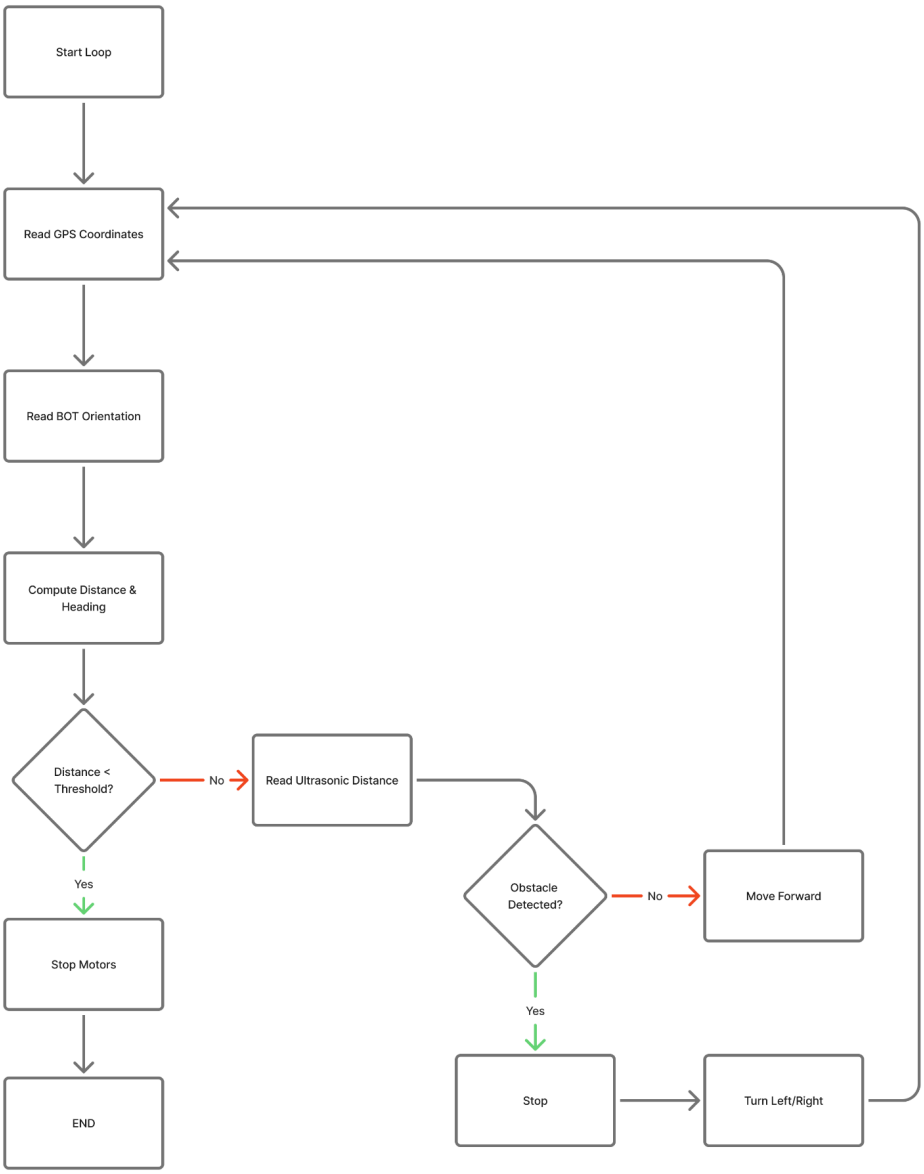
This chapter explains the step-by-step operation of the GPS Enabled Autonomous Food Delivery Bot and the algorithm used for navigation and obstacle avoidance.

5.1 Working Principle

The robot operates based on continuous feedback from the GPS module, magnetometer, and ultrasonic sensor. The destination coordinates are provided by the user through a mobile application.

Step-by-Step Operation

1. **System Power ON:** The robot is powered using 18650 batteries and the buck converter stabilizes voltage for all components.
2. **Wi-Fi Initialization:** The ESP32 establishes a Wi-Fi connection with the mobile application.
3. **Destination Input:** The user sets the destination latitude and longitude in the mobile app.
4. **GPS Acquisition:** The GPS module starts receiving satellite signals and provides current location data.
5. **Position Comparison:** The ESP32 compares the current coordinates with the target coordinates.
6. **Direction Determination:** Using magnetometer data, the robot determines its heading and required direction of movement.
7. **Motor Actuation:** The motor driver receives control signals and drives the motors to move the robot.
8. **Obstacle Detection:** The ultrasonic sensor continuously checks for obstacles.
9. **Obstacle Avoidance:** If an obstacle is detected, the robot changes direction until the path is clear.
10. **Destination Reached:** When the robot enters a predefined radius (for example, within 5 meters of the target), it stops.
11. **Food Collection:** The packed food container can be manually collected from the robot.



5.2 Navigation Algorithm

The navigation algorithm is based on distance comparison and directional correction.

Algorithm Steps

1. Read current GPS coordinates
2. Calculate difference between current and target coordinates
3. Determine required heading angle
4. Compare with current orientation (from magnetometer)
5. Adjust motor speeds accordingly

6. Check obstacle distance
7. If obstacle detected → perform avoidance maneuver
8. Repeat until destination is reached

5.3 Obstacle Avoidance Algorithm

A simple reactive obstacle avoidance approach is used.

Logic

- If distance > threshold → move forward
- If distance ≤ threshold:
    - Stop
    - Turn right by 90 degrees
    - Move forward for few seconds
    - Turn left by 90 degrees
    - Align to the original path
    - Continue forward

This allows the robot to safely navigate around objects.

CHAPTER 6 – ADVANTAGES, APPLICATIONS, AND LIMITATIONS

6.1 Advantages

The GPS Enabled Autonomous Food Delivery Bot offers several benefits:

- **Reduced Human Effort:** Minimizes dependency on manual delivery personnel.
- **Contactless Delivery:** Suitable for hospitals and restricted areas.
- **Cost-Effective Design:** Uses low-cost, easily available components.
- **Autonomous Operation:** Navigates independently using GPS and sensors.
- **Scalable Architecture:** Can be extended with cameras, AI, or cloud connectivity.
- **Energy Efficient:** Operates on rechargeable batteries.

6.2 Applications

This system can be deployed in:

- Restaurants and Cafes
- College and University Campuses

- Hospitals and Healthcare Facilities
- Industrial Campuses
- Warehouses
- Smart Cities

It is ideal for short-distance autonomous delivery in controlled outdoor environments.

6.3 Limitations

Despite its advantages, the system has certain limitations:

- GPS accuracy is affected in indoor or covered areas
- Performance degrades in crowded environments
- Limited battery life restricts operating time
- Obstacle avoidance is reactive, not predictive
- Navigation accuracy depends on satellite visibility

These limitations can be addressed through future enhancements.

CHAPTER 7 – FUTURE SCOPE

The GPS Enabled Autonomous Food Delivery Bot demonstrates the feasibility of low-cost autonomous delivery. However, the system can be significantly enhanced in the future through the following improvements:

- **AI-Based Path Planning:** Integration of advanced algorithms for optimal route selection and dynamic path optimization.
- **Camera-Based Navigation**
  Use of computer vision for lane detection, object recognition, and precise localization.
- **Indoor Navigation Support:** Incorporation of technologies such as RFID, BLE beacons, or SLAM for indoor environments.
- **Two-Way Communication:** Live status updates to the mobile application, including robot position and battery level.
- **Voice Interaction:** Integration of voice commands for hands-free operation.
- **Solar Charging System:** Addition of solar panels to extend operational time and reduce dependency on manual charging.
- **Load Management:** Sensors to detect container weight and adjust speed and power usage accordingly.

These enhancements can transform the prototype into a commercial-grade autonomous delivery platform.

CHAPTER 8 – CONCLUSION

The **GPS Enabled Autonomous Food Delivery Bot** successfully demonstrates an efficient and reliable solution for short-distance autonomous delivery. The system integrates GPS-based navigation, real-time obstacle avoidance, wireless communication, and motor control into a compact embedded platform.

By using a passive food container, the prototype remains mechanically simple while effectively showcasing autonomous transportation. The robot can receive destination coordinates from a mobile application, navigate toward the target, avoid obstacles, and stop at the destination without human intervention.

The experimental results validate the design objectives and confirm the feasibility of low-cost autonomous delivery systems for campuses, hospitals, and industrial environments. With future enhancements such as AI-based navigation and camera integration, this system can be scaled for real-world commercial deployment.